

SmartMeter Blink Developer Guide

Version 1.0 — internal architecture and build information.

1. Overview

A single-screen Jetpack Compose Android app that talks to a Landis+Gyr/EYKON E320 smart meter through light pulses. The whole UI is rendered by `MainScreen.kt` and driven by one `BlinkViewModel` exposing a `MutableStateFlow<UiState>`. All flashing logic is in the `ViewModel`; physical output is abstracted behind a `LightSource` (torch or full-white screen flash).

Property	Value
Package / applicationId	de.smartmeter.blink
minSdk / targetSdk	26 / 35
compileSdk	release(37) — AGP 9 DSL, see build
Kotlin / Compose	Kotlin 2.2.10, Compose BOM 2026.02.01 (Material 3)
Dependencies	activity-compose, compose ui/material3/graphics/tooling, core-ktx, lifecycle runtime + viewmodel-android. No appcompat, no navigation, no Room.

2. Building

Requirements

- Android SDK with platform `android-37` and build-tools `36.x`.
- JDK 25 (the Android Studio JBR is used: `/home/petric/android-studio/jbr`).
- Gradle wrapper `9.5.0` / AGP `9.3.2` in the repo (gradlew, configuration cache is on).

Commands

```
export JAVA_HOME=/home/petric/android-studio/jbr
./gradlew :app:assembleDebug
./gradlew :app:assembleRelease

# Install on a connected device / emulator
$ANDROID_HOME/platform-tools/adb install -r app/build/outputs/apk/debug/SmartMeter-Blink.apk
```

Signed installable APK: `app/build/outputs/apk/release/SmartMeter-Blink.apk`
Debug APK: `app/build/outputs/apk/debug/SmartMeter-Blink.apk`. The build tasks also emit `app-debug.apk/app-release.apk` in the same folders — the `SmartMeter-Blink.apk` names above are the ones kept for sharing/installing.

Signing

Release builds are signed with a dedicated release keystore. The keystore file and its passwords are never committed to the repository. Credentials are read at build time from environment variables (`BLINK_RELEASE_KEYSTORE`, `BLINK_STORE_PASSWORD`, `BLINK_KEY_ALIAS`, `BLINK_KEY_PASSWORD`) or from the local, `chmod-600` file `~/.android/blink-release.properties`. Create the key once:

```
keytool -genkeypair -noprompt \
  -keystore ~/.android/blink-release.keystore -alias blink \
  -storepass '<secret>' -keypass '<secret>' \
  -keyalg RSA -keysize 4096 -validity 10950 \
  -dname "CN=SmartMeter Blink, O=SmartMeter Blink, C=DE"

# ~/.android/blink-release.properties (chmod 600)
blinkReleaseKeystore=/home/<you>/.android/blink-release.keystore
blinkStorePassword=<secret>
blinkKeyAlias=blink
blinkKeyPassword=<secret>
```

If no credentials are configured, the build falls back to the debug keystore (~/.android/debug.keystore, alias androiddebugkey) with a warning — fine for sideloading, never for publishing. Back up the keystore and passwords: losing them means you can no longer update an installed app. minification is off (isMinifyEnabled = false).

minSdk is 26 because the app uses an adaptive launcher icon.

3. Project Structure

app/src/main/	
AndroidManifest.xml	permissions + main activity
java/de/smartmeter/blink/	
MainActivity.kt	ComponentActivity, sets Compose content
BlinkViewModel.kt	state + all flashing/sequence logic
FlashlightController.kt	CameraManager.setTorchMode wrapper + lastError
LightSource.kt	LightSourceMode + torch/screen sources
MeterTiming.kt	timing defaults & ranges
MeterProfile.kt	profile model + predefined-profile loader
QuickAction.kt	inF / Pin actions + menu model
ui/	
MainScreen.kt	the whole UI (profiles, PIN pad, cards, settings)
theme/	Color.kt, Type.kt, Theme.kt
assets/	
meter_profiles.json	predefined meter configs (format:1 JSON)
res/	
values/strings.xml	English strings (source of truth)
values-de/strings.xml	German strings
build.gradle.kts	

4. Architecture & State

BlinkViewModel

- Extends `AndroidViewModel`; owns a `CoroutineScope(SupervisorJob() + Dispatchers.Main.immediate)`.
- Exposes `UiState` (data class) via `StateFlow`: pin, light source, torch availability/error, dark-screen & wake flags, profile list + active id, running flag, active digit index, strobe flag, current quick action, progress fraction and progress text.
- `runOperation(label, estimatedTotalMs, block)` is the central runner: cancels any previous job, marks the state running, runs the suspend block with a `Progress` (report) helper, then in finally turns the light off, vibrates a pattern and resets the state. The caller can `stop()` at any time, which cancels job — all pulsing code is `delay()`-based and interruption leaves the light off.
- The `Progress` inner class accumulates reported milliseconds and renders "... · ~Ns left" from the estimated total (`estimatePin()` precomputes the PIN part; the quick-action total adds wake + scroll + long-hold).

Physical output

- `LightSourceMode = TORCH | SCREEN`. The `ViewModel` keeps a pending state field and turns the light on/off via `currentLight()`:
 - `TORCH` → `TorchLightSource` → `FlashlightController.setTorch`.
 - `SCREEN` → `ScreenLightSource` → sets `UiState.strobe`, which `MainScreen.kt` renders as a full-white overlay (`Color.White`).
- `FlashlightController` resolves the flash camera lazily and records a localized `lastError` instead of throwing (`SecurityException` → permission hint, `CameraAccessException` → busy, no flash → not available). The camera permission is requested once at app start in the composable via `rememberLauncherForActivityResult`.

5. Sequence Logic

Protocol (E320)

```
menu indices after unlock (short flash = 1 step, start at P):
  0 P   1 E   2 E CLr  3 1d  4 7d  5 30d  6 365d  7 HiS CLr  8 inF  9 Pin

* one short flash from P wakes the meter into the display test (~4 s), then the PIN prompt
* digit d is sent as d flashes (0 = no flash / wait)
* after each digit except the last the app waits digitWait (default 3.4 s)
* a long flash (>=4s) on inF / Pin toggles it – NEVER on CLr items (erases data)
```

Send PIN

`wakeMeter()` (optional, default on) = 1 short pulse + `wakeWaitMs` wait, `sendPinDigits()` = per digit: report, flash loop (on=pulseLength, off=pulseGap), accept-wait. Length is guarded by `appendDigit()` using `profile.pinLength`.

Quick actions

`runQuickAction(action, sendPinFirst)`: wake → (PIN or skip) → scroll `opticalScrolls/`
`pinScrolls` short pulses (gap between) → one long pulse. The `sendPinFirst` flag is a *per-run* choice because when the meter shows `Pin OFF` every flash scrolls the menu — sending PIN digits would walk past the target. If PIN is required but incomplete, a dialog (composed in the UI) asks for it.

6. Persistence & Data Formats

SharedPreferences ("settings")

Key
light_source
dark_screen / wake_meter
profiles
active_profile
profile_name_<id>, profile_pinlen_<id>, profile_pulse_<id>, profile_gap_<id>, profile_digit_<id>, profile_pin_scrolls_<id>

On first launch the predefined profiles from `assets/meter_profiles.json` are loaded via `MeterProfile.loadPredefined(...)` (parsed once with the `platform org.json`, then cached) and seeded as the initial profile list. In case the asset is missing or invalid the app falls back to `MeterProfile.DEFAULT` so it never starts without a profile. A localized description (`descriptionRes`) is re-attached using the `meter_profile_<id>_desc` resource convention. Descriptions are not serialized to JSON as resources, but plain description text is.

JSON format (export/import)

```
[ { "format": 1, "id": "e320", "name": "Landis+Gyr/EYKON E320", "pinLength": 4,
  "pulseLengthMs": 180, "pulseGapMs": 320, "digitWaitMs": 3400, "longPulseMs":
  6000,
  "opticalScrolls": 8, "pinScrolls": 9, "description": "..." } ]
```

Implemented with the `platform org.json` classes (no extra dependency). Import accepts an array or a single object, merges by `id` (known → update, unknown → add, invalid → skip), clamps every numeric field to its valid range, and re-persists the list. Export writes via SAF (`CreateDocument/OpenDocument` launchers in the UI) to get a user-chosen location.

7. Localization

- All user-facing strings live in `res/values/strings.xml` (English, source of truth) and `res/values-de/strings.xml` (German) and are resolved automatically from the **device** display language.
- Composables use `stringResource(R.string.x)`; callbacks that need a resolved string at click-time pre-resolve it with `stringResource` and use `String.format(Locale.ROOT, ...)` for format args.
- `ViewModel` messages use `getApplication<Application>().getString(resId, ...)` via a private `str()` helper; `FlashlightController` uses its stored application context; `QuickAction.label(context)` and `MeterProfile.descriptionRes` cover the remaining dynamic text.

8. Permissions

Permission	Why
<code>android.permission.CAMERA</code>	<code>setTorchMode</code> throws without it; requested at runtime on first launch.
<code>android.permission.VIBRATE</code>	Haptic feedback/pulse pattern at the end of a run. (Also declared as non-required features <code>camera + camera.flash</code> .)

9. Code Layout Notes

- AGP 9 DSL quirks in `app/build.gradle.kts`: `compileSdk { version = release(37) }`, `optimization { enable = false }`, and signing via `signingConfigs.create("release")`.
- No `viewmodel-compose` artifact is used: the activity holds the `ViewModel` via `viewModels()` and passes it into the root composable explicitly.
- Orientation: the manifest no longer locks `screenOrientation`, so the activity rotates into landscape on its own. No `configChanges` handling is used — the activity is recreated (state survives via `rememberSaveable` and the `ViewModel`).

- `MainScreen.kt` is intentionally one file; its private composables are `ProfileSelector`, `KeypadButton`, `QuickActionCard`, `SettingsCard`, `LanguagePicker`-style helpers (none left), `TimingSliderRow` and `NumberStepper`.
- The dark-run overlay (`state.running`) is a black full-screen box that also captures taps → `viewModel.stop()`; the strobe overlay is drawn above it for the screen light source.

10. Possible Improvements

- Richer localization (more languages, plural rules for the ETA seconds).
- Instrumented tests for the sequence logic (currently no unit tests for the timing math).
- Accessibility: content descriptions for the keypad and torch controls.

Landis+Gyr, EYKON and E320 are trademarks of their owners. This app is an independent hobby tool.

This app and its source code may not be used to obtain a meter PIN by brute force or to gain unauthorized access to a meter.